

Practical : 4	Implementation and Time analysis of Factorial program using Iterative and Recursive method.
Date : 12/08/2026	

Objective :- To implement the Factorial program in C++ using both the Iterative and Recursive methods, and to analyse and compare their time complexity through practical execution-time measurements for varying values of n.

Program :-

```
#include <iostream>
#include <vector>
#include <chrono>
using namespace std;
using namespace std::chrono;

const long long MOD = 1000000007; // keep results bounded for large n

// ----- Iterative Factorial -----
long long factorialIterative(int n) {
    long long result = 1;
    for (int i = 2; i <= n; i++)
        result = (result * i) % MOD;
    return result;
}

// ----- Recursive Factorial -----
long long factorialRecursive(int n) {
    if (n <= 1) return 1;
    return (n * factorialRecursive(n - 1)) % MOD;
}

// ----- Timing helper -----
const int TRIALS = 2000; // repeat & average for a measurable, stable time

template <typename Func>
double measureAvgTime(Func f, int n) {
    volatile long long sink = 0;
    auto start = high_resolution_clock::now();
    for (int t = 0; t < TRIALS; t++)
        sink += f(n);
    auto stop = high_resolution_clock::now();
    (void)sink;
    return duration_cast<duration<double, micro>>(stop - start).count() / (double)TRIALS;
}

int main() {
    vector<int> sizes = {10, 100, 1000, 5000, 10000, 20000};
```

```

    cout << "n\tIterative(avg us)\tRecursive(avg us)\n";
    for (int n : sizes) {
        double tIter = measureAvgTime(factorialIterative, n);
        double tRec = measureAvgTime(factorialRecursive, n);
        cout << n << "\t" << tIter << "\t\t" << tRec << "\n";
    }
    return 0;
}

```

Analysis:-

Time & Space Complexity Comparison:

Method	Time Complexity	Space (Auxiliary)	Stack Depth
Iterative	$O(n)$	$O(1)$	$O(1)$
Recursive	$O(n)$	$O(n)$ (call stack)	$O(n)$

Observed Execution Time for computing $n!$ (mod 1,000,000,007 to keep values bounded for large n). Each figure is the average of 2000 repeated runs to obtain a stable, measurable reading:

n	Iterative (avg of 2000 runs, μs)	Recursive (avg of 2000 runs, μs)
10	0.0186	0.0189
100	0.420	0.441
1000	4.396	4.570
5000	22.690	22.856
10000	45.635	45.888
20000	91.522	92.297

Both methods perform exactly $n-1$ multiplications, so both run in $O(n)$ time — the growth in the observed timings is linear in n for both columns, roughly a 5000x increase in time as n goes from 10 to 20000 (a 2000x increase in n). The Recursive method is consistently a little slower than the Iterative method at every n , since each recursive call carries function-call overhead (pushing a new stack frame, saving return address and local state) that the Iterative version avoids entirely by using a single loop variable. The more significant difference is in space: Iterative factorial uses $O(1)$ auxiliary space regardless of n , while Recursive factorial uses $O(n)$ auxiliary space for the call stack, since n nested calls must remain active until the base case is reached and the multiplications unwind. This also means the recursive version risks stack overflow for very large n , a failure mode the iterative version does not have.

Conclusion :- Both the Iterative and Recursive factorial methods were successfully implemented and their execution times measured across increasing values of n , confirming that both run in $O(n)$ time. The Iterative method is marginally faster in practice due to the absence of function-call overhead, and crucially uses only $O(1)$ auxiliary space compared to the Recursive method's $O(n)$ stack space. This makes Iterative factorial the more efficient and robust choice for large n , while the Recursive method remains valuable for its conceptual clarity and as a foundation for understanding recursion, divide-and-conquer, and call-stack behaviour in more complex algorithms.